My first 90 days as Lexroom's first Analytics Engineer

Mattia Franzè, October 2026

Point of view. The first analytics hire is not paid to write SQL; they are paid to make a handful of numbers trustworthy enough that Product, CS, Sales and Engineering stop arguing about them. At Lexroom today, ~1,000 users across IT, DE and ES, 15+ modules, >100k AI artifacts a week landing from Pub/Sub into BigQuery, and no transformation layer yet, that means: start where decisions are already being made on shaky numbers, build the smallest thing that fixes one of them, and let every next step be pulled by a question someone actually asked. My background is a large warehouse migration programme; the discipline this role needs in the first 90 days is the opposite of that, and I know it.

1. Priorities: 30 / 60 / 90 days

Days 1 to 30: learn how data is born, ship one trusted number.

- Map how events are produced and land: which services emit which Pub/Sub topics, how schemas change, what happens on retries. The case data already shows what to expect: duplicates, orphan keys, out-of-domain values, inconsistent timestamps.
- Day one in production: `dbt source freshness` and daily row-count checks on the landing tables. Cheap, and the first test that catches a real problem.
- One question to each team: "which number do you not trust today?" That list is the roadmap.
- First mart: AMA quality, because Product, CS and Leadership all read it from different angles. One intermediate model holding the definitions, three marts, docs a PM can read, tests in CI on every PR.

Days 31 to 60: second and third domains, pulled by questions.

- Likely candidates: workspace usage and activation (CS and Sales both need it, from the same table), module adoption per plan and country (Product). I will not know for sure until day 30, and that is the point.
- A definitions page that lives next to the models (dbt docs, surfaced where people already look), and exposures linking each mart to the question it answers.
- A 15-minute weekly slot with one named referent per team. Not a committee: a channel for "this number looks wrong".

Days 61 to 90: ownership, self-service, baseline SLAs.

- Every mart gets a business owner (a person, not a team) and a technical owner (me).
- A thin self-service layer on top of the marts, in whatever BI tool already exists. Only on marts whose definitions have survived 30 days of use.
- SLAs committed only after 30 days of measured freshness and volume.
- A review with the CTO: what gets built next, and what gets stopped.

2. Data quality framework

Tests that would catch a real problem, in three tiers, and nothing else:

- **Sources:** freshness, row-count anomalies, known upstream issues as `warn` so they are visible on every run without blocking it.
- **Keys and grain:** uniqueness on the declared grain, relationships on the joins that feed a decision.
- **Reconciliation:** singular tests that prove the marts add up to the inputs (every event counted once, rates between 0 and 1, breakdowns summing to totals). These are the ones that catch fan-out and silent filters, the two bugs that quietly move every KPI.

Without slowing anyone down:

- Tests run in CI on every pull request, in seconds, on a DuckDB or dev dataset. A failing test blocks the merge; a warning does not.
- Severity is a decision: `error` only when a wrong number would reach someone who acts on it.
- Boilerplate (YAML, basic tests, docs stubs) is generated with AI tooling, so engineering time goes to definitions and edge cases, not typing.
- **Ownership** is a lightweight contract with Engineering: event schema changes go through a review I am part of, and in return they get a test that tells them when an event they produce breaks downstream, before a stakeholder does.
- **SLAs** come last and come from data: a baseline first, then one SLA per mart, owned jointly with the engineer who produces the source.

3. Trust and self-service, team by team

- **Product** wants trends and reasons, at a grain that survives experiments: module × week, with the "why" behind a thumbs down. Trust is earned when the mart answers their question in one short query and the definition is written where they read it. Self-service: the weekly module mart in the BI tool, with `is_low_sample` visible so nobody acts on 12 ratings.
- **Customer Success** wants a list of accounts to call, with a reason per account. Trust is earned by not crying wolf: usage signals relative to the platform trend, "not enough data" kept separate from "healthy", a drop confirmed in two windows before it tops the list. Self-service: one table, one row per workspace, refreshed daily.
- **Sales** wants usage and adoption per workspace and plan for renewals and upsell, and will compare them with what CS sees. Trust is earned when both teams read the same row. Self-service: the same workspace mart, with plan and seat data joined, surfaced in the CRM if that is where they live.
- **Engineering** is my upstream, not my customer. Trust is earned by reporting problems with evidence (the query, the rows, the impact) and by never asking for ad hoc extracts. What they get back: a data contract on the events that matter, and alerts that catch schema drift before anyone else does.

Self-service in that order: definitions, then marts, then a thin BI layer, then training. The reverse order produces dashboards that disagree.

4. What I would not do, and why scope discipline matters

- **No migration programme.** Not every table into dbt; only the domains with a question behind them. A programme makes sense with a team and a multi-year horizon; alone, reporting to a CTO who needs results, it means six months without a visible number.
- **No macros or frameworks before the third repetition.** The shared incremental framework I wrote at lastminute.com came after seeing the same pattern ten times. With 5 to 10 models, a macro is overhead.
- **No dashboards for everyone before definitions are stable.** Dashboards built on moving definitions are how trust is lost.
- **No SLAs without a baseline.** A promise without a measurement is a promise you break.
- **No becoming the query desk.** Ad hoc requests are accepted when they produce a reusable model; otherwise they get a definition and a query the requester can run.

Why it matters here: one person, four consumer teams, a product that ships continuously. Every hour on infrastructure nobody asked for is an hour not spent making one number trustworthy. In 90 days credibility comes from a few numbers people stopped arguing about, not from a well-structured repo nobody reads.

5. The biggest risks

1. **Definitions diverge before they have a home.** With 15 modules and three teams already computing their own numbers, "active user" may already mean three things. Mitigation: one intermediate layer holding definitions, docs a PM reads, and the definitions page as the place disagreements get settled.
2. **Serving the CTO instead of the teams.** I report to Engineering; my customers are Product, CS and Sales. If priorities come only from the reporting line, I build infrastructure that nobody uses. Mitigation: the weekly referent slot, and the "which number do you not trust" list as the shared backlog.
3. **The product changes faster than the models.** 100k artifacts a week, continuous releases, no schema contracts on events yet. Mitigation: freshness and anomaly tests on sources, `warn` tests on known issues, and the schema-change review agreed with Engineering.
4. **Bus factor of one.** Everything I build is tacit knowledge unless I make it otherwise. Mitigation: docs and exposures from day one, and one engineer who can run and extend the project by day 90.
5. **Over-building, my own bias.** I come from a large, well-staffed migration. The mitigation is the list in section 4, and a CTO who holds me to it.

Day 90, if this works: three marts with named owners, tests in CI, one definitions page people link to in arguments, and a backlog written by the teams rather than by me.